Riset Operasi

Modifikasi Vogel's Approximation Method (VAM)

Erica Triana Widyastuti
5025241069

Dilbina Windi Azahra
5025241180

Isabella Sienna Sulisthio
5025241199

Nasyita Larasati Ertyananda
5025241201

Rahma Sakinah
5025241222

November 10, 2026

1. Langkah VAM Original

Algoritma **Vogel's Approximation Method (VAM)** klasik bekerja berdasarkan prinsip penalti (opportunity cost) yang dihitung dari selisih dua elemen biaya terkecil pada setiap baris dan kolom.

1. Cek total suplai (ΣS_i) dan total permintaan (ΣD_j).
2. Hitung Penalti Tiap Baris dan Kolom Aktif
   a. Untuk setiap baris aktif i, cari biaya terendah pertama ($c_i^{(1)}$) dan kedua ($c_i^{(2)}$).
   Penalti baris: $P_i = c_i^{(2)} - c_i^{(1)}$
   b. Untuk setiap kolom aktif j, cari biaya terendah pertama ($c_j^{(1)}$) dan kedua ($c_j^{(2)}$).
   Penalti kolom: $P_j = c_j^{(2)} - c_j^{(1)}$
3. Pilih Garis dengan Penalti Terbesar
   a. Tentukan baris atau kolom dengan nilai penalti maksimum, yaitu $max(P_i, P_j)$.
   b. Aturan tie-breaking: Jika terdapat penalti bernilai sama, implementasi standar memilih kandidat baris/kolom pertama yang ditemukan.
4. Pilih Sel Biaya Minimum ($min(c_{ij})$) pada baris atau kolom terpilih.
5. Alokasikan Kuantitas Transportasi
   a. Hitung nilai alokasi: $q = min(S_i, D_j)$
   b. Kurangi suplai dan permintaan berdasarkan alokasi:
      - $S_i = S_i - q$
      - $D_j = D_j - q$
   c. Jika $S_i = 0$, baris i dieliminasi dari iterasi berikutnya.
   d. Jika $D_j = 0$, kolom j dieliminasi dari iterasi berikutnya.
6. Ulangi langkah 2 sampai 5 hingga seluruh kapasitas suplai dan demand habis teralokasi.

2. Langkah VAM Modifikasi (SERDA)

SERDA Method memodifikasi perhitungan penalti dan seleksi sel menggunakan deviasi terhadap rata-rata baris dan kolom aktif (*Dynamic Mean Opportunity*) serta menerapkan hierarki *tie-breaking* deterministik:

1. Cek total suplai (ΣS_i) dan total permintaan (ΣD_j).
2. Hitung Rata-rata Biaya Baris & Kolom Aktif:
   a. Rata-rata baris aktif : $AVG_i = \frac{\sum_{j \in J} C_{ij}}{|J|}$, dengan J adalah jumlah kolom aktif.
   b. Rata-rata kolom aktif : $AVG_j = \frac{\sum_{i \in I} C_{ij}}{|I|}$, dengan I adalah jumlah baris aktif.
3. Hitung biaya relatif (*CR*) untuk setiap sel aktif (i, j): $CR_{ij} = C_{ij} - AVG_i - AVG_j$.
4. Hitung Penalti Tiap Baris dan Kolom dari biaya relatif (CR_{ij}) :
   a. Untuk setiap baris CR_i, cari biaya terendah pertama ($CR_i^{(1)}$) dan kedua ($CR_i^{(2)}$). Penalti baris: $P_i = CR_i^{(2)} - CR_i^{(1)}$
   b. Untuk setiap kolom CR_j, cari biaya terendah pertama ($CR_j^{(1)}$) dan kedua ($CR_j^{(2)}$). Penalti kolom: $P_j = CR_j^{(2)} - CR_j^{(1)}$
5. Pilih Garis dengan Penalti terbesar. Jika terjadi *Tie*, maka urutan *tie breaker*-nya:
   a. Prioritas 1: Penalti Terbesar
   b. Prioritas 2 (*Tie*): Nilai biaya relative terkecil ($min(CR_{ij})$)
   c. Prioritas 3 (*Tie*): Baris diprioritaskan disbanding kolom
   d. Prioritas 4 (*Tie*): Indeks terkecil
6. Pada baris atau kolom terpilih, pilih Sel $min(CR_{ij})$. Jika *Tie*, maka urutan *tie breaker*-nya:
   a. Prioritas 1: Nilai biaya relative terkecil ($min(CR_{ij})$)
   b. Prioritas 2 (*Tie*): Biaya Transportasi Asli terkecil ($min(c_{ij})$)
   c. Prioritas 3 (*Tie*): Indeks terkecil
7. Alokasikan Kuantitas Transportasi
   a. Hitung nilai alokasi: $q = min(S_i, D_j)$
   b. Kurangi suplai dan permintaan berdasarkan alokasi:
      - $S_i = S_i - q$
      - $D_j = D_j - q$
   c. Jika $S_i = 0$, baris i dieliminasi dari iterasi berikutnya.
   d. Jika $D_j = 0$, kolom j dieliminasi dari iterasi berikutnya.
8. Ulangi langkah 2 sampai 7 hingga seluruh kapasitas suplai dan demand habis teralokasi.

3. Perbandingan Kode

Pseudocode VAM Original

```
function solve_VAM(C, supply,
      demand):
  balance_problem(C, supply,
      demand)

  while supply_left > 0:
    // 1. Hitung Penalti Biaya
      Asli
    for each active row i:
      P[i] = second_min(C[i]) -
      min(C[i])
    for each active col j:
      Q[j] = second_min(C[:][j])
      - min(C[:][j])

    // 2. Pilih Penalti Terbesar
    line = max(P, Q)

    // 3. Pilih Sel Biaya
      Minimum
    if line is row:
      j = argmin_{j in
      active_cols}(C[line][j])
      sel = (line, j)
    else:
      i = argmin_{i in
      active_rows}(C[i][line])
      sel = (i, line)

    // 4. Alokasikan Unit
    q = min(supply[sel.i],
      demand[sel.j])
    alloc[sel.i][sel.j] += q
    supply[sel.i] -= q
    demand[sel.j] -= q
    eliminate_zero_lines()

return alloc, total_cost
```

Pseudocode VAM Modif (SERDA)

```
function solve_SERDA(C, supply, demand):
  balance_problem(C, supply, demand)

  while supply_left > 0:
    // 1. Hitung Rata-rata Baris & Kolom
      Aktif
    for each active row i:
      row_mean[i] =
      mean(C[i][active_cols])
    for each active col j:
      col_mean[j] =
      mean(C[active_rows][j])

    // 2. Transformasi CR & Penalti CR
    for each active row i:
      CR[i][*] = C[i][*] - row_mean[i] -
      col_mean[*]
      P[i] = second_min(CR[i]) -
      min(CR[i])
      min_cr_row[i] = min(CR[i])
    for each active col j:
      CR[*][j] = C[*][j] - row_mean[*] -
      col_mean[j]
      Q[j] = second_min(CR[:][j]) -
      min(CR[:][j])
      min_cr_col[j] = min(CR[:][j])

    // 3. Seleksi Garis Terbaik (Tie-break
      4 Tingkat)
    line = select_best_line(
             P, Q, min_cr, priority:
             1. Max Penalty
             2. Min CR terkecil
             3. Baris sebelum kolom
             4. Indeks terkecil
           )

    // 4. Seleksi Sel Terbaik (Tie-break 3
      Tingkat)
    sel = select_best_cell(
            line, priority:
            1. Min CR
            2. Min Original Cost C[i][j]
            3. Indeks terkecil
          )

    // 5. Alokasikan Unit
    q = min(supply[sel.i], demand[sel.j])
    alloc[sel.i][sel.j] += q
    supply[sel.i] -= q
    demand[sel.j] -= q
    eliminate_zero_lines()
return alloc, total_cost
```

4. Implementasi Kode

Link Github: https://github.com/IsabellaSienna01/SERDA-Method

SERDA.c > ...

```c
TPSolution* solve_SERDA(TPProblem *prob, int verbose) {
    while (1) {

        /* Row candidates */
        for (int ri = 0; ri < n_active_r; ri++) {
            int i = active_r[ri];
            for (int cj = 0; cj < n_active_c; cj++) {
                int j = active_c[cj];
                cr_vals[cj] = prob->cost[i][j] - row_mean[i] - col_mean[j];
            }
            qsort(cr_vals, n_active_c, sizeof(double), cmp_double);
            double penalty = (n_active_c >= 2) ? (cr_vals[1] - cr_vals[0]) : 0.0;
            lines[n_lines].is_row = 1;
            lines[n_lines].idx = i;
            lines[n_lines].penalty = penalty;
            lines[n_lines].min_cr = cr_vals[0];
            n_lines++;
        }

        /* Column candidates */
        for (int cj = 0; cj < n_active_c; cj++) {
            int j = active_c[cj];
            for (int ri = 0; ri < n_active_r; ri++) {
                int i = active_r[ri];
                cr_vals[ri] = prob->cost[i][j] - row_mean[i] - col_mean[j];
            }
            qsort(cr_vals, n_active_r, sizeof(double), cmp_double);
            double penalty = (n_active_r >= 2) ? (cr_vals[1] - cr_vals[0]) : 0.0;
            lines[n_lines].is_row = 0;
            lines[n_lines].idx = j;
            lines[n_lines].penalty = penalty;
            lines[n_lines].min_cr = cr_vals[0];
            n_lines++;
        }

        /* 3. Sort lines by tie-breaking rule */
        qsort(lines, n_lines, sizeof(LineCandidate), cmp_line_candidates);
```

Figure 1: Potongan kode implementasi solve SERDA

5. Perbandingan Hasil

VAM Original		VAM Modif (SERDA)		Solusi Optimal
Test Case	**Cost Solution**	**Test Case**	**Cost Solution**	**Cost Solution**
G1_N01	5600*	G1_N01	5600*	5600
G1_N12	425*	G1_N12	425*	425
G1_N21	4205*	G1_N21	4205*	4205
G1_P07	327	G1_P07	291*	291
G1_P11	600*	G1_P11	600*	600
G1_P17	490	G1_P17	475*	475
G1_P18	555*	G1_P18	555*	555
G1_P19	1500	G1_P19	1390*	1390
G2_N02	28*	G2_N02	28*	28
G2_N14	960	G2_N14	920*	920
G2_N15	156*	G2_N15	156*	156
G2_N22	12075*	G2_N22	12075*	12075
G2_N23	23*	G2_N23	23*	23
G2_P12	180*	G2_P12	180*	180
G3_N31	1480*	G3_N31	1480*	1480
G3_N33	1430	G3_N33	1250*	1250
G3_N35	320	G3_N35	280*	280
G3_P02	2850*	G3_P02	2850*	2850
G3_P03	149*	G3_P03	149*	149
G3_P04	450*	G3_P04	450*	450
G3_P09	845*	G3_P09	845*	845
G3_P13	3210*	G3_P13	3210*	3210
G3_P20	3320*	G3_P20	3320*	3320
G3_S07	326	G3_S07	305*	305
G4_N07	1220	G4_N07	1165	1160
G4_N10	990	G4_N10	910*	910
G4_N11	2980	G4_N11	2460*	2460
G4_N16	1590	G4_N16	1510*	1510
G4_N17	779	G4_N17	743*	743
G4_N25	849	G4_N25	809*	809
G4_N29	3857*	G4_N29	3857*	3857
G4_P01	605*	G4_P01	605*	605
G5_N06	8350	G5_N06	7750*	7750

G5_N13	5125	G5_N13	4525*	4525
G5_N18	1745	G5_N18	1650*	1650
G5_N24	3520	G5_N24	3460*	3460
G5_N27	68804	G5_N27	59356*	59356
G5_N28	2090*	G5_N28	2090*	2090
G5_N30	473	G5_N30	465*	465
G6_N04	1580*	G6_N04	1580*	1580
G6_N09	114	G6_N09	111*	111
G6_N32	1820	G6_N32	1820	1780
G6_N34	510*	G6_N34	510*	510
G6_P05	290*	G6_P05	290*	290
G6_P10	212*	G6_P10	212*	212
G6_RBSM1	18855*	G6_RBSM1	18855*	18855
G6_RBSM2	153500*	G6_RBSM2	153500*	153500
G6_S04	3150	G6_S04	3150	3125
G7_N03	1104	G7_N03	1103	1102
G7_N05	470	G7_N05	415	410
G7_N26	490*	G7_N26	502	490
G7_P06	49*	G7_P06	49*	49
G7_P08	11000	G7_P08	10625*	10625
G7_P14	175*	G7_P14	175*	175
G7_P15	12810	G7_P15	10830*	10830
G7_P16	340	G7_P16	325*	325
G8_RBSM3	1044180*	G8_RBSM3	1044180*	1044180
G8_RBSM4	93351	G8_RBSM4	92259	92212
G8_RBSM5	116020*	G8_RBSM5	116020*	116020
G8_S01	5148	G8_S01	5302	5067
G8_S02	1089	G8_S02	1098	1080
G8_S03	948	G8_S03	969	885
G8_S05	1801	G8_S05	1600	1586
G8_S06	5409	G8_S06	5022	4702
G9_R1	12928429*	G9_R1	12928429*	12928429
G9_R2	13657053*	G9_R2	13657053*	13657053
G9_R3	12296587*	G9_R3	12296587*	12296587
G9_R4	14117586*	G9_R4	14117586*	14117586
G9_R5	14017419*	G9_R5	14017419*	14017419